

PRODUCT REQUIREMENTS DOCUMENT — v3.0

Rozgar Manager

Expense · Sales · Distribution · Inventory · Routes · Order Takers

 **Desktop
(Electron)**

 **Web (Next.js)**

 **SQLite +
MongoDB**

 **Admin-Controlled**

 **Live DB Switch**

v3 — Admin Access Model · Live DB Switching · Auto-Migration · Local Admin Panel

Version 3.0 | March 2026 | Confidential

1. What's New in Version 3.0

Area	V2.0	V3.0 Enhancement
Admin DB Access (SQLite)	Admin logs into web admin panel	Admin physically sits at PC, logs into embedded Admin Mode inside Electron app, accesses .db files locally
Admin DB Access (MongoDB)	Admin Panel at /admin route	Admin logs into /admin from anywhere — no physical access needed
DB Mode Change	Not fully defined	Only admin can change DB_MODE per user. Change takes effect immediately (live switch) without app restart
Data on DB Switch	Manual migration described	System auto-migrates all data on switch — SQLite → MongoDB and MongoDB → SQLite both supported
Auto-migration trigger	Not defined	Triggered by admin changing DB_MODE in either Admin Mode (local) or Admin Panel (web)
Master Admin credential	Not defined	Hidden master password baked into app build. Not visible to any user. Used to unlock Admin Mode on any PC.

2. Admin Identity & Access Model

There is one super-admin: you. You are not a regular user of the app. You have two separate entry points depending on whether the client is on SQLite (desktop) or MongoDB (web/cloud). These are completely different interfaces with the same powers.

Property	SQLite Client	MongoDB Client
How you access	Physically sit at client PC (or remote desktop)	From anywhere via browser
Entry point	Hidden Admin Mode inside Electron app	Web Admin Panel at /admin route
Credential type	Master Admin Password (baked into build)	ADMIN_SECRET env variable
Internet required	✗ No — fully offline	✓ Yes — web panel
What you can do	See all .db files, manage users, change DB mode, trigger migration	See all tenants/users, manage accounts, change DB mode, trigger migration
Can change DB_MODE	✓ Yes — only you	✓ Yes — only you
Regular user can change DB_MODE	✗ Never	✗ Never

👑 **ADMIN ONLY:** DB_MODE is the single most protected setting in the app. It is not in any user-facing settings screen. It is only accessible after authenticating as the master admin.

3. Admin Mode — SQLite / Local (On-Site Access)

Admin Mode is a hidden overlay inside the Electron app that you unlock when physically present at a client's PC. It is completely invisible to regular users — there is no button, no menu item, no hint it exists.

3.1 How to Unlock Admin Mode

Step	Action	Detail
1	Sit at client PC	App is running normally, client is logged out or you ask them to step away
2	Trigger hidden shortcut	Press Ctrl + Shift + Alt + A (configurable key combo, not documented to users)
3	Master password prompt appears	A minimal modal appears: single password field, no branding, no hints
4	Enter Master Admin Password	Password is hashed and baked into the Electron build at compile time
5	Admin Mode overlay opens	Full admin dashboard slides in over the app — client UI is locked behind it
6	Do your work	Manage users, change DB mode, trigger migration, view .db files
7	Exit Admin Mode	Press Escape or click Exit — overlay closes, normal app resumes

 **SECURITY:** The Master Admin Password is set by you at build time via an environment variable (MASTER_ADMIN_HASH). It is bcrypt-hashed before being baked in. Even if someone decompiles the app, they only see the hash, never the plain password.

3.2 Admin Mode — Screen Layout

Screen	What You See & Can Do
Dashboard	All .db files on this machine, file sizes, last login per user, app version, DB_MODE status
User Management	List of all local users from db_registry.json. Create user, reset password, deactivate, delete user + .db file
DB Mode Control	Current DB_MODE for this installation. Button to switch: SQLite ↔ MongoDB. Triggers auto-migration on confirm.
Database Files	File browser showing all .db files in databases\ folder. View file size, last modified, open in hex (debug), export .db
Migration Tool	Manual trigger for SQLite → MongoDB or MongoDB → SQLite migration with progress bar and error log
Export & Backup	Trigger full JSON/CSV/db export for any or all users. Choose destination folder.
App Config	View/edit app.config.json — DB_MODE, MongoDB URI, AppProfile preset, app name, version

Screen	What You See & Can Do
Audit Log	All admin actions logged: who, what, when. Cannot be deleted by regular users.

3.3 Admin Mode — DB_MODE Change Flow (SQLite → MongoDB)

When you change a user's DB_MODE from SQLite to MongoDB inside Admin Mode, the following happens immediately and automatically:

```

Admin clicks 'Switch to MongoDB' for user ahmed@company.com
↓
System prompts: Enter MongoDB URI for this user (or use default from app.config)
↓
Admin confirms — auto-migration begins immediately
↓
Step 1: Read user's .db file (user_ahmed.db) → extract all tables to memory
↓
Step 2: Connect to target MongoDB URI
↓
Step 3: Map SQLite schema → MongoDB schema (see Section 6)
↓
Step 4: Insert all records into MongoDB with tenantId stamped on each document
↓
Step 5: Verify record counts match (SQLite rows = MongoDB documents per collection)
↓
Step 6: If verification passes → update app.config.json: DB_MODE = 'mongodb'
↓
Step 7: Archive (do NOT delete) user's .db file → rename to
user_ahmed_archived_YYYY-MM-DD.db
↓
Step 8: Live switch — app immediately connects to MongoDB for this user
↓
User experience: if they were logged in, they see a brief 'Switching database...' spinner
then the app reloads their data from MongoDB seamlessly

```

⚠ WARNING: The old .db file is NEVER deleted automatically. It is archived with a date suffix. You or the client can delete it manually after confirming migration was successful.

3.4 Admin Mode — DB_MODE Change Flow (MongoDB → SQLite)

```

Admin clicks 'Switch to SQLite' for user ahmed@company.com
↓
System warns: This will download all MongoDB data to local .db file
Admin confirms — auto-migration begins immediately
↓
Step 1: Connect to MongoDB, fetch all documents for tenantId
↓
Step 2: Create new .db file: user_ahmed.db (encrypted with SQLCipher)
↓
Step 3: Map MongoDB schema → SQLite schema
↓

```

```
Step 4: Insert all records into new .db file
↓
Step 5: Verify counts match
↓
Step 6: Update app.config.json: DB_MODE = 'sqlite'
↓
Step 7: Live switch — app immediately reads from local SQLite
```

4. Admin Panel — MongoDB / Remote (Access from Anywhere)

For MongoDB clients, you manage everything through the web-based Admin Panel. You can be in Karachi, the client can be in Lahore — you log into the Admin Panel from your browser and do everything remotely.

4.1 Admin Panel Access

Property	Detail
URL	yourdomain.com/admin (or separate admin.yourdomain.com)
Authentication	ADMIN_SECRET environment variable — single master password you set on the server
Visible to users	✗ No — /admin route is not linked from anywhere in the app UI
Session	Admin session separate from user sessions, 2-hour expiry, IP-locked (optional)
Access from	Any browser, any device, any location — just needs internet
2FA	Optional TOTP (Google Authenticator) — recommended for production

4.2 Admin Panel — What You Can Do

Feature	Description
Tenant Management	Create, view, edit, deactivate all client businesses (tenants)
User Creation	Create user accounts for any tenant. Set name, email, password, role. No self-signup ever.
DB Mode Control	Change DB_MODE per user or per tenant. Takes effect immediately. Auto-migration runs on confirm.
Password Reset	Reset any user's password, mark forcePasswordChange for next login
Account Deactivation	Deactivate user (can't login) or delete user + all their data
AppProfile Assignment	Set or change business type preset (distributor, shop, FMCG, wholesale) per tenant
Migration Tool	SQLite JSON import → MongoDB tenant. MongoDB → SQLite export package.

Feature	Description
Data Export	Export any tenant's data as JSON or CSV ZIP. Download to your machine.
Audit Log	Every admin action logged: timestamp, action, affected user/tenant
System Health	MongoDB connection status, storage usage per tenant, active sessions, error logs

👑 **ADMIN ONLY:** DB_MODE change in Admin Panel works identically to Admin Mode in Electron — immediate live switch with auto-migration. The user sees a brief spinner then continues working in the new DB.

5. DB Mode Control — Complete Rules

5.1 Who Can Do What

Action	Owner	Manager	Salesman	Accountant	Viewer	Admin (You)
View DB_MODE setting	✗	✗	✗	✗	✗	✓
Change DB_MODE	✗	✗	✗	✗	✗	✓
Create user accounts	✓ (local only)	✗	✗	✗	✗	✓ Full
Trigger migration	✗	✗	✗	✗	✗	✓
Export data	✓ Own data	✓ Own data	✗	✓ Own data	✗	✓ Any user
Access Admin Mode / Panel	✗	✗	✗	✗	✗	✓ Only
View other users' data	✗	✗	✗	✗	✗	✓ Admin only

5.2 DB Mode Change — Decision Tree

You want to change a user's DB_MODE

Is the user on SQLite (desktop)?

YES → You must physically be at their PC (or remote desktop)

→ Open Electron app

→ Ctrl + Shift + Alt + A → Enter Master Admin Password

→ Admin Mode opens → DB Mode Control → Select user → Switch

→ Auto-migration runs immediately → Live switch

Is the user on MongoDB (web)?

YES → Open browser from anywhere

→ Go to yourdomain.com/admin → Login with ADMIN_SECRET

→ Tenant Management → Select user → DB Mode → Switch

→ Auto-migration runs immediately → Live switch

In BOTH cases:

- User sees 'Switching database...' spinner if currently logged in
- After 5-30 seconds: app reloads with new DB, all data intact
- Old data source is archived, never deleted automatically

6. Auto-Migration Engine

The Auto-Migration Engine is a background service that runs when any DB_MODE change is confirmed by the admin. It handles both directions: SQLite → MongoDB and MongoDB → SQLite. It is the most complex component in the system.

6.1 Migration Phases

Phase	Name	Action	On Failure
1	Pre-flight Check	Verify source DB is readable. Verify target DB is reachable. Check disk space (SQLite) or connection (MongoDB).	Abort — show error. No data touched.
2	Snapshot	Create a point-in-time snapshot of source data. For SQLite: copy .db to .db.migration_snapshot. For MongoDB: export to temp JSON in memory.	Abort — no data touched.
3	Schema Map	Map all tables/collections through the schema translation layer (see 6.2).	Abort — log unmapped fields.
4	Transform	Convert data types, rename fields, add tenantId (SQLite→Mongo), flatten/expand documents.	Abort — rollback snapshot.
5	Write	Insert all transformed records into target DB in batches of 500.	Pause — retry 3x — then rollback.
6	Verify	Compare record counts: source vs target per table/collection. Must be 100% match.	Rollback target writes — restore from snapshot.
7	Switch	Update app.config.json DB_MODE. Reconnect DBAdapter to new target.	Rollback all — user stays on original DB.
8	Archive	Rename source to _archived_YYYY-MM-DD (SQLite .db file or MongoDB collection prefix).	Non-critical — log warning only.
9	Notify	Show admin success screen with stats. If user was online, show them 'Migration complete' toast.	N/A

6.2 Schema Translation Map

SQLite Table	MongoDB Collection	Key Transformations
users	users	Add tenantId. Passwords stay bcrypt-hashed. Add mongold field.
app_config	tenants	Merge into tenant document as config subdocument.
products	products	Add tenantId. INTEGER ids → ObjectId. Add timestamps.
shops	shops	Add tenantId. routeld INTEGER → routeld ObjectId ref.
routes	routes	Add tenantId. shopIds array of INTEGERS → ObjectId refs.
invoices	invoices	Add tenantId. Embed invoice_items as items[] array subdoc.
invoice_items	(embedded)	Merged into invoices.items[]. No separate collection.
expenses	expenses	Add tenantId. attachedTo string → attachedTo {type, id} object.
payments	payments	Add tenantId. invoiceld INTEGER → ObjectId ref.
stock_movements	stockMovements	Add tenantId. snake_case → camelCase field names.
suppliers	suppliers	Add tenantId. Standard field rename only.
purchase_orders	purchaseOrders	Add tenantId. Embed purchase_order_items as items[].

i NOTE: invoice_items and purchase_order_items are separate tables in SQLite (relational). In MongoDB they become embedded arrays inside their parent document. This is the most complex transformation and is handled by the migration engine automatically.

6.3 Migration Progress UI

While migration runs, both the admin and the logged-in user (if any) see a non-dismissible progress overlay:

Element	Detail
Progress bar	0–100% based on phases completed
Phase label	Shows current phase name: 'Verifying source...', 'Transforming data...', etc.
Record counter	Live count: 'Migrating invoices: 847 / 1,203'
Time estimate	Estimated time remaining based on current batch rate
Cancel button	Available only during Phase 1–3. Disabled once writes begin (Phase 5+).
Error display	If any phase fails, red error box with specific failure reason and rollback status
Success screen	Green checkmark, record counts per table, time taken, archive file path

6.4 Rollback Guarantee

✓ If migration fails at ANY phase after Phase 4, the system automatically rolls back. The user's data is restored to exactly the state it was before migration started. They never lose data due to a failed migration.

- SQLite rollback: delete partial target, restore from .db.migration_snapshot
- MongoDB rollback: delete all documents written in this migration session (tracked by migrationSessionId)
- app.config.json DB_MODE is ONLY updated after Phase 6 verification passes
- All rollback actions are logged to admin audit log

7. User Account Creation — Complete Rules

7.1 The Golden Rule

🔒 **SECURITY:** There is NO self-signup anywhere in this app. Not on web, not on desktop. Every single user account — without exception — is either created by you (admin) or by the bootstrap script seeded by you.

7.2 All Account Creation Paths

Path	When	Who Initiates	How
Bootstrap Mode A (SQLite)	First app run, seed.config.json present	You (pre-configured before delivery)	seed.config.json auto-processed by bootstrap.js
Bootstrap Mode B (SQLite)	First app run, no seed file	Client (with your knowledge)	Setup wizard — creates owner account only
Admin Mode (SQLite)	Any time after first run	You (physically at PC)	Admin Mode → User Management → Create User
Owner adds user (SQLite)	After first run, owner role only	Client's owner account	Settings → User Management → Add User (creates new .db)
Admin Panel (MongoDB)	Any time	You (from anywhere)	Admin Panel → Tenant → Users → Create User
Admin Mode (MongoDB via Electron)	If Electron + MongoDB mode	You (physically at PC)	Admin Mode → User Management (connects to MongoDB)

⚠ **WARNING:** Owner adding a user in SQLite mode is allowed but limited — the owner can only create accounts for their own installation (their db_registry). They cannot change DB_MODE, access admin features, or see other tenants.

7.3 Account Creation — Field Spec

Field	Required	Rules
Full Name	✓	2–80 characters, display name only
Email	✓	Unique per installation (SQLite) or per tenant (MongoDB). Used as login identifier.
Password	✓	Min 8 chars, at least 1 number. bcrypt-hashed before storage. Never stored plain.
Role	✓	owner manager salesman accountant viewer
Force Password Change	Optional	If true, user must change password on first login before accessing app
DB_MODE assignment	Admin only	Set by admin only. Not shown on user creation form for non-admin.
AppProfile preset	Admin / Owner	Business type that controls which modules this user sees
isActive	Auto	Defaults true. Admin or owner can set false to block login without deleting account.

8. Bootstrap System (Updated v3)

The bootstrap system is the SQLite account provisioning mechanism. It runs inside the Electron main process before any window opens. It is silent and near-instantaneous when users already exist.

8.1 Complete Bootstrap Flow

```

Electron app starts
↓
main.js: await bootstrap() ← runs before BrowserWindow created
↓
bootstrap.js:
  1. Read AppData/BizCoreManager/app.config.json
     → If missing: create with defaults (DB_MODE: 'sqlite')
  2. Read db_registry.json
     → If registry has 1+ users: EXIT BOOTSTRAP (skip to login)
  3. Registry empty – check seed.config.json

— MODE A: seed.config.json EXISTS —
  → Validate JSON structure
  → For each user in seed.users:
    a. Hash password with bcrypt (cost 12)
    b. Create databases/user_<email_slug>.db
    c. Run schema migrations on new .db
    d. Insert user record into .db
    e. Register in db_registry.json
  → if deleteAfterSeed: true → securely wipe seed.config.json
  → LOG: 'Bootstrap Mode A: N users created'
  → PROCEED to Login Screen

```

```
— MODE B: seed.config.json MISSING —  
→ LOG: 'Bootstrap Mode B: showing setup wizard'  
→ PROCEED to First-Run Setup Wizard  
    (user creates their own owner account)  
    (after wizard: creates .db, writes registry, goes to login)
```

8.2 seed.config.json — Full Spec

```
{  
  "version": "1.0",  
  "deleteAfterSeed": true,  
  "appProfile": "distributor",  
  "dbMode": "sqlite",  
  "users": [  
    {  
      "name": "Ahmed Khan",  
      "email": "ahmed@bizcore.com",  
      "password": "Welcome@2026",  
      "role": "owner",  
      "forcePasswordChangeOnLogin": true  
    },  
    {  
      "name": "Sara Ali",  
      "email": "sara@bizcore.com",  
      "password": "Staff@2026",  
      "role": "salesman",  
      "forcePasswordChangeOnLogin": false  
    }  
  ]  
}
```

 **SECURITY:** seed.config.json with deleteAfterSeed: true is cryptographically wiped (overwritten with zeros, then deleted) — not just deleted. A forensic recovery tool cannot recover the plain-text passwords from disk.

9. Data Export System

9.1 Export Formats

Format	Contents	Use Case	Who Can Trigger
Full .db Copy	Byte-for-byte copy of user's SQLite database file (encrypted)	Complete backup, restore to new PC, disaster recovery	Owner, Admin
JSON Export	All tables serialized to structured JSON. Human-readable. Includes schema version.	MongoDB migration, developer inspection, API import, cross-system transfer	Owner, Accountant, Admin

Format	Contents	Use Case	Who Can Trigger
CSV ZIP	One .csv file per table, packaged into a single .zip archive	Excel analysis, accountant review, financial audit, reporting	Owner, Accountant, Admin
MongoDB export (Mongo mode)	JSON export of tenant's MongoDB data (no raw DB access)	Backup, migration to SQLite, data portability	Admin only

9.2 Export Options

- Export All Data — every table, all history, no date filter
- Export by Date Range — invoices, expenses, payments within chosen period
- Export Selected Modules — pick: Sales only, Expenses only, Inventory only, etc.
- Auto-Export Schedule — daily/weekly auto-export to a user-chosen local folder, USB, or network path
- Export Destination — user selects any folder on their PC or external drive

9.3 Import & Restore

Operation	What It Does	Who Can Do It
Restore from .db	Replaces current .db file entirely. Requires confirmation. Old .db archived first.	Admin (via Admin Mode)
Import from JSON	Merges or replaces data from JSON export. User chooses: merge (keep existing + add new) or replace (wipe and reimport).	Admin (via Admin Mode)
Restore from CSV	Not supported for full restore. CSV is export-only format.	N/A
Move to new PC	Copy .db file + db_registry.json to new machine's AppData folder. Bootstrap detects registry and skips setup.	User (with guidance) or Admin

10. Security Model

Threat	Mitigation
User discovers Admin Mode shortcut	Shortcut key combo is not documented, not shown in UI, not in help docs. Can be changed per build.
User brute-forces Master Admin Password	3 wrong attempts → 60-second lockout. 10 attempts → app closes. Rate limiting in Electron main process.
User reads seed.config.json to get plain passwords	deleteAfterSeed: true wipes file cryptographically on first run. File does not exist after bootstrap.

Threat	Mitigation
User decompiles app to find Master Admin Password	Only bcrypt hash is stored in build. Hash alone cannot be reversed. Salt is randomized per build.
User copies another user's .db file	Each .db is encrypted with SQLCipher AES-256. Without the correct key (derived from user credentials), file is unreadable.
User modifies app.config.json to change DB_MODE	app.config.json is read-only from within the app UI. Manual edits are detected via checksum on next launch — triggers integrity warning.
Admin Panel brute-force (MongoDB)	Rate limiting: 5 attempts → 15-min lockout. Optional IP whitelist. Optional 2FA (TOTP).
MongoDB connection string exposed	Stored in server environment variable, never in client-side code or localStorage.
Migration data interception	All MongoDB connections use TLS/SSL. SQLite migration data processed in-memory, never written to temp files in plaintext.

11. Updated Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 + React 18	Shared web/desktop UI
UI Components	shadcn/ui + Tailwind CSS	Role-filtered component rendering
Desktop Shell	Electron (latest stable)	Windows packaging, IPC, file system
SQLite Driver	better-sqlite3 + SQLCipher	Encrypted local .db files per user
MongoDB Driver	Mongoose 8 + MongoDB Node Driver	Cloud/VPS data, tenant middleware
DB Abstraction	Custom DBAdapter class	Single interface — routes to SQLite or MongoDB
Auth (Web)	NextAuth.js v5	MongoDB session management, role middleware
Auth (Desktop)	Local JWT + Electron safeStorage	Offline auth, encrypted token storage
Bootstrap	bootstrap.js in Electron main process	Pre-window account provisioning
Admin Mode	Electron IPC + hidden overlay component	Admin-only overlay, keyboard shortcut trigger
Master Password	bcrypt hash in build (env var at compile time)	Admin Mode authentication
Migration Engine	Custom MigrationService class	Bidirectional SQLite ↔ MongoDB migration
Password Hashing	bcrypt (cost factor 12)	All passwords in both DB modes
SQLite Encryption	SQLCipher AES-256	Full .db file encryption at rest
PDF Generation	React-PDF	Invoices, reports, challans
Export Service	Custom ExportService	.db copy, JSON serializer, CSV/ZIP builder

Layer	Technology	Purpose
State Management	Zustand	Client-side reactive state
Packaging	electron-builder + NSIS	Windows .exe installer
Auto-Update	electron-updater	Delta updates from your update server
Audit Logging	Custom AuditLogger	All admin actions logged, tamper-evident

12. Development Roadmap (v3 Updated)

Phase	Weeks	Key Deliverables
0 — Foundation	1–2	Project structure, DB Adapter interface, SQLite schema, MongoDB schema, build pipeline
1 — Bootstrap + Auth	3–5	bootstrap.js, seed.config system, setup wizard, local JWT auth, NextAuth, db_registry
2 — Admin Mode	6–7	Hidden shortcut, Master Admin Password system, Admin Mode overlay, user management UI
3 — Core Modules	8–12	Sales/invoicing, expense tracking, inventory, shop management, PDF generation
4 — Distribution	13–15	Routes, order takers, role permissions, advanced inventory, reports
5 — Migration Engine	16–17	MigrationService, all 9 phases, rollback system, progress UI, schema translation map
6 — Admin Panel (Web)	18–19	MongoDB Admin Panel /admin, tenant management, DB mode control, audit log
7 — Export System	20	.db copy, JSON/CSV export, auto-export scheduler, import/restore
8 — Security + Polish	21–22	SQLCipher integration, rate limiting, brute-force protection, checksum integrity, QA
9 — Ship	23–24	Electron packaging, NSIS installer, auto-updater, Windows signing, final QA
Post-launch	Ongoing	Mobile app, Urdu UI, WhatsApp integration, FBR tax, analytics, Mac/Linux builds

13. Remaining Open Questions

#	Question	Options	Recommendation
1	Admin Mode shortcut key	Ctrl+Shift+Alt+A / Custom combo / Click hidden UI area	Ctrl+Shift+Alt+A — simple, not discoverable by accident
2	Master Admin Password rotation	Fixed per build / Changeable from Admin Mode	Changeable from Admin Mode — rotate every few months

#	Question	Options	Recommendation
3	LAN multi-user SQLite	Single PC only / Share .db over network	Single PC for MVP — LAN adds significant complexity
4	MongoDB hosting default	Atlas / Contabo VPS / Client-managed	Contabo VPS — cheap, fast, Pakistan-friendly latency
5	Electron: code signing	Self-signed / Windows Authenticode cert	Authenticode cert (~\$100/yr) — prevents SmartScreen warning on install
6	Urdu UI	MVP / Phase 2 / Never	Phase 2 — biggest differentiator for local market